

Función comerciar:

Pre: Las dos ciudades existen, son diferentes y los identificadores de los productos están ordenados de manera creciente.

Post: Las dos ciudades comercian entre ellas. Una ciudad le dará a la otra todos los productos que le sobren hasta alcanzar, si es posible, los que la otra necesite, y viceversa.

Invariante: el iterador it1 está entre `this->inventario_productos.begin()` y `this->inventario_productos.end()` y el iterador it2 está entre `ciudad_2.inventario_productos.begin()` y `ciudad_2.inventario_productos.end()`.

Es decir, `this->inventario_productos.begin() <= it1 <= this->inventario_productos.end()` y `ciudad_2.inventario_productos.begin() <= it2 <= ciudad_2.inventario_productos.end()`.

```
void Ciudad::comerciar(Ciudad& ciudad_2, const Productos& productos) {
    map<int, pair<int, int>>::iterator it1 = this->inventario_productos.begin();
    map<int, pair<int, int>>::iterator it2 = ciudad_2.inventario_productos.begin();
    while (it1 != this->inventario_productos.end() and it2 !=
ciudad_2.inventario_productos.end()) {

        int id_prod1 = it1->first;
        int id_prod2 = it2->first;

        if (id_prod1 > id_prod2) ++it2;
        else if (id_prod1 < id_prod2) ++it1;
        else {

            int sobrante1 = it1->second.first - it1->second.second;
            int sobrante2 = it2->second.first - it2->second.second;

            int peso_prod = productos.consultar_peso(id_prod1);
            int volumen_prod = productos.consultar_volumen(id_prod1);

            if (sobrante1 > 0 and sobrante2 < 0) { // solo hacemos intercambio si le
sobran productos a la ciudad_1 y necesita productos a la ciudad 2
                int intercambio = min(sobrante1, -sobrante2);
                it1->second.first -= intercambio; // se decrementan las unidades en
posesión de la ciudad 1
                it2->second.first += intercambio; // se incrementan las unidades en
posesión de la ciudad 2

                // Actualizar el peso y el volumen total
                this->peso_total -= intercambio * peso_prod;
                this->volumen_total -= intercambio * volumen_prod;
                ciudad_2.peso_total += intercambio * peso_prod;
                ciudad_2.volumen_total += intercambio * volumen_prod;
```

```

    }
    else if (sobrante1 < 0 and sobrante2 > 0) { // solo hacemos intercambio si
necesita productos a la ciudad_1 y le sobran productos a la ciudad 2
        int intercambio = min(-sobrante1, sobrante2);
        it1->second.first += intercambio; // se incrementan las unidades en
posesión de la ciudad 1
        it2->second.first -= intercambio; // se decrementan las unidades en
posesión de la ciudad 2

        // Actualizar el peso y el volumen total
        this->peso_total += intercambio * peso_prod;
        this->volumen_total += intercambio * volumen_prod;
        ciudad_2.peso_total -= intercambio * peso_prod;
        ciudad_2.volumen_total -= intercambio * volumen_prod;
    }
    ++it1;
    ++it2;
}
}
}

```

Justificación de la función comerciar:

Paso 1 (inicialización): Los iteradores se inicializan al primer elemento del map de su inventario de productos, es decir, a `.begin()`. Por lo tanto, cumplen la invariante porque si `.begin()` es diferente de `.end()` se cumple la invariante y si en algun map el `.begin()` = `.end()` entonces también se cumple la invariante, además las ciudades aún no han comerciado entre ellas.

Paso 2 (condición de salida): Si alguno de los dos iteradores llega al final de su map, es decir, a `.end()`, entonces quiere decir que ya hemos explorado todos los elementos del inventario de productos de una de las ciudades y que las ciudades habrán comerciado entre ellas cuando hayan podido. También cabe recalcar que si la invariante es cierta antes de una iteración, sigue siendo cierta después de cada iteración, porque la condición del bucle es que los iteradores sean diferentes de `.end()`. Por lo tanto, en el peor de los casos, al acabar una iteración, uno de los dos iteradores, o los dos, apuntarán a `.end()`, donde en este caso se sigue cumpliendo la invariante.

Paso 3 (cuerpo del bucle): Si el iterador 1 tiene un identificador de producto más grande que el iterador 2, entonces se avanza el iterador 2. Si el identificador del iterador 1 es más pequeño que el del iterador 2, entonces se avanza el iterador 1. Si ninguna de las condiciones anteriores se cumple, significa que los dos iteradores tienen un mismo identificador y, por lo tanto, las ciudades pueden llegar a comerciar si se cumplen las condiciones adecuadas para que comercien entre ellas. En el caso de que los iteradores tengan el mismo identificador del producto, avanzamos los dos iteradores en una posición y seguimos iterando.

Paso 4 (condición de terminación): La condición de terminación del bucle es que los iteradores sean iguales a `.end()`. Por lo tanto, el bucle hace iteraciones desde `.begin()` de los dos maps hasta que uno de los dos llegue a `.end()`. En cada iteración se aumenta uno de los iteradores, por lo tanto, en cada iteración disminuye la distancia entre los iteradores y el `map<int, pair<int,int>>.size()` de alguno de los dos maps. Esto nos indica que la función siempre termina porque la distancia disminuye hasta cero, que es cuando termina el bucle

Función hacer viaje:

Pre: Cierto

Post: Se obtiene la mejor ruta que puede hacer el barco entre las diferentes ciudades y luego se realiza la mejor ruta obtenida, comerciando entre las diferentes ciudades y el barco. También se escribe el total de unidades de productos compradas y vendidas por el barco, y se añade a las ciudades visitadas por el barco la última ciudad donde se ha hecho algún tipo de intercambio.

Hipótesis de inducción: Al hacer la llamada a esta función, nos devuelve el número de intercambios máximos que se pueden hacer entre el barco y las ciudades, así como la ruta por la que hay que pasar para obtener este número de intercambios máximos.

```
int Cuenca::mejor_ruta(list<string>& ruta, const BinTree<string>& a, int
id_prod_comprar, int id_prod_vender, pair<int, int> prods_barco, map<string,
Ciudad>& ciudades) {
    if (a.empty()) {
        ruta = list<string> ();
        return 0;
    }
    else {
        string id_ciudad = a.value();
        int intercambios = 0;

        if (ciudades[id_ciudad].prod_existe(id_prod_comprar)) {
            pair<int, int> p = ciudades[id_ciudad].consultar_prod(id_prod_comprar);
            // devuelve un pair donde el primer elemento son las uds_obtenidas y el segundo las
            uds_necesitadas
            int sobrante = p.first - p.second;
                // sobrante = uds_en_posesión - uds_necesitadas
            if (sobrante > 0) {
                // se compra productos para el barco solo si nos sobran productos en la ciudad
                int aux = min(sobrante, prods_barco.first);
                intercambios += aux;
                // añadimos a los intercambios el sobrante
                prods_barco.first -= aux;
            }
        }

        if (ciudades[id_ciudad].prod_existe(id_prod_vender)) {
            pair<int, int> p = ciudades[id_ciudad].consultar_prod(id_prod_vender);
            // devuelve un pair donde el primer elemento son las uds_obtenidas y el segundo las
            uds_necesitadas
            int faltante = p.second - p.first;
                // faltante = uds_necesitadas - uds_en_posesion
            if (faltante > 0) {
                // se venden productos del barco solo si nos faltan productos en la ciudad
```

```

        int aux = min(faltante, prods_barco.second);
        intercambios += aux;
// añadimos a los intercambios el faltante
        prods_barco.second -= aux;
    }
}

list<string> l1;
list<string> l2;
int intercambios1 = mejor_ruta(l1, a.left(), id_prod_comprar, id_prod_vender,
prods_barco, ciudades);
int intercambios2 = mejor_ruta(l2, a.right(), id_prod_comprar, id_prod_vender,
prods_barco, ciudades);

if (intercambios1 == 0 and intercambios2 == 0 and intercambios == 0) {
    ruta = list<string> ();
    return 0;
}

else if (intercambios1 > intercambios2) {
    ruta = l1;
    ruta.push_front(a.value());
    return intercambios1 + intercambios;
}
else if (intercambios1 < intercambios2) {
    ruta = l2;
    ruta.push_front(a.value());
    return intercambios2 + intercambios;
}
else {
    if (l1.size() <= l2.size()) {
        ruta = l1;
        ruta.push_front(a.value());
        return intercambios1 + intercambios;
    }
    else {
        ruta = l2;
        ruta.push_front(a.value());
        return intercambios2 + intercambios;
    }
}
}
}

```

Justificación de la función recursiva hacer viaje:

Paso 1 (caso sencillo): Si el árbol es vacío, entonces no tiene ninguna ciudad y, por lo tanto, el barco no puede comerciar. Como no se puede comerciar con ninguna ciudad y además no tiene subárboles, devolvemos que el número de intercambios es 0 y una lista vacía porque no se ha visitado ninguna ciudad.

Paso 2 (caso recursivo): Si el árbol no es vacío, entonces el barco puede llegar a comerciar con la ciudad en la que estamos. Por lo tanto, vamos a comprobar si el barco puede llegar a comerciar con la ciudad en la que estamos y calcular el número de intercambios que se han hecho con la ciudad. El número de intercambios totales se obtiene por hipótesis de inducción con las llamadas recursivas, que son correctas y cumplen la precondition. Por lo tanto, obtenemos el número de intercambios del subárbol izquierdo con las ciudades visitadas y el número de intercambios en el subárbol derecho con las ciudades visitadas. De este modo, podemos obtener cuál es la mejor ruta y además el número máximo de intercambios que se pueden hacer comparando los dos subárboles izquierdo y derecho, así obtendremos la ruta con la cual se puede obtener este número máximo de intercambios y por lo tanto se cumple la postcondición.

Paso 3 (decrecimiento): Cada vez que se hace una llamada recursiva a la función, estamos accediendo a árboles más pequeños, porque estamos accediendo a los subárboles izquierdo y derecho. Con lo cual, llegará un punto en el que no habrá más subárboles derecho o izquierdo y se dará el caso base donde nos encontramos en un árbol vacío. De tal manera que cada vez el tamaño del árbol va disminuyendo y, como consecuencia, la función llega al caso base.